

Pedirle cosas a los datos

¿Dónde ganó Morena por más de veinte puntos?

Emiliano Miranda González

Instituto Tecnológico Autónomo de México
Departamento Académico de Ciencia Política

Sesión 3 de 11

ITAM, otoño 2026

¿POR QUÉ NOS IMPORTA ESTO?

¿Dónde ganó Morena por más de veinte puntos?

Tienes una intuición: que Morena arrasó en el sur y compitió parejo en otras partes. En esta hora esa intuición se vuelve una línea de código que contesta con una tabla, no con una sensación.

De dónde venimos

- La sesión pasada abrimos una base electoral de verdad.

De dónde venimos

- La sesión pasada abrimos una base electoral de verdad.
- Vimos qué es una fila y qué es una columna, con `glimpse()`.

De dónde venimos

- La sesión pasada abrimos una base electoral de verdad.
- Vimos qué es una fila y qué es una columna, con `glimpse()`.
- Aprendimos a leer el tipo de cada columna y a desconfiar de un `NA`.

De dónde venimos

- La sesión pasada abrimos una base electoral de verdad.
- Vimos qué es una fila y qué es una columna, con `glimpse()`.
- Aprendimos a leer el tipo de cada columna y a desconfiar de un `NA`.
- Hoy le vamos a `pedir cosas` a esa tabla.

Por qué no usamos corchetes

`filter()`: de una sensación a una tabla

La trampa silenciosa

`select()`, `arrange()` y sus atajos

`mutate()` y la escala municipal

Ejercicio

Cómo filtrábamos antes del pipe

```
resultados[resultados$participacion > 60,  
             c("entidad", "participacion")]
```

El nombre de la tabla se repite tres veces. Un paréntesis o una coma fuera de lugar y todo truena, sin decirte dónde.

La misma pregunta, con el pipe

```
resultados |>  
  filter(participacion > 60) |>  
  select(entidad, participacion)
```

Cada verbo dice lo que hace. Se lee de arriba hacia abajo, como una receta.

Esto es exactamente lo que hacía el bloque anterior: ninguno de los dos modifica **resultados**.

El pipe se lee «y luego»

- `|>` toma lo que sale de un bloque y lo pasa al siguiente.

El pipe se lee «y luego»

- `|>` toma lo que sale de un bloque y lo pasa al siguiente.
- Se escribe con `Ctrl+Shift+M` (`Cmd+Shift+M` en Mac).

El pipe se lee «y luego»

- `|>` toma lo que sale de un bloque y lo pasa al siguiente.
- Se escribe con `Ctrl+Shift+M` (`Cmd+Shift+M` en Mac).
- En este curso nunca vamos a usar `%>%`: es el pipe viejo, de antes de que R trajera uno de fábrica en 2021.

El pipe se lee «y luego»

- `|>` toma lo que sale de un bloque y lo pasa al siguiente.
- Se escribe con `Ctrl+Shift+M` (`Cmd+Shift+M` en Mac).
- En este curso nunca vamos a usar `%>%`: es el pipe viejo, de antes de que R trajera uno de fábrica en 2021.

Vas a escribir `|>` más veces que cualquier otro símbolo del curso. Vale la pena que el atajo te salga sin pensar.

Por qué no usamos corchetes

`filter()`: de una sensación a una tabla

La trampa silenciosa

`select()`, `arrange()` y sus atajos

`mutate()` y la escala municipal

Ejercicio

Quedarte con las filas que cumplen algo

```
resultados |> filter(entidad == "Jalisco")  
  
resultados |> filter(participacion > 60)
```

`==` pregunta si dos cosas son iguales. `=` sirve para otra cosa
—asignar— y dentro de `filter()` casi siempre es un error de dedo.

Combinar condiciones

```
resultados |>  
  filter(pct_shh > 60 | pct_fcm > 60)
```

- ==, !=, >, <, >=, <= comparan.

Combinar condiciones

```
resultados |>  
  filter(pct_shh > 60 | pct_fcm > 60)
```

- ==, !=, >, <, >=, <= comparan.
- & exige que se cumplan las dos condiciones.

Combinar condiciones

```
resultados |>  
  filter(pct_shh > 60 | pct_fcm > 60)
```

- ==, !=, >, <, >=, <= comparan.
- & exige que se cumplan las dos condiciones.
- | exige que se cumpla al menos una.

Por qué no usamos corchetes

`filter()`: de una sensación a una tabla

La trampa silenciosa

`select()`, `arrange()` y sus atajos

`mutate()` y la escala municipal

Ejercicio

Un filtro que «funciona» y miente

- Imagina una columna con el código de la coalición ganadora: 1 para Sigamos Haciendo Historia, 2 para Fuerza y Corazón por México.

Un filtro que «funciona» y miente

- Imagina una columna con el código de la coalición ganadora: 1 para Sigamos Haciendo Historia, 2 para Fuerza y Corazón por México.
- Alguien escribe `filter(codigo == 1 | 2)` para quedarse con esas dos.

Un filtro que «funciona» y miente

- Imagina una columna con el código de la coalición ganadora: 1 para Sigamos Haciendo Historia, 2 para Fuerza y Corazón por México.
- Alguien escribe `filter(codigo == 1 | 2)` para quedarse con esas dos.
- R no truena. Te entrega una tabla. Esa tabla está mal.

Compruébalo en la consola

```
codigo_coalicion <- c(1, 2, 3, 1, 2)  
codigo_coalicion == 1 | 2
```

Este vector es un ejemplo didáctico, no un dato real. Corre las dos líneas y mira lo que devuelve la segunda.

Por qué pasa esto

```
codigo_coalicion == 1 | 2 no marca ningún error
```

R primero calcula `codigo_coalicion == 1`. Después le hace un `OR` con el número 2, y R convierte cualquier número distinto de cero en `TRUE`.

El resultado es `TRUE` en todas partes. Con `filter()`, eso significa quedarte con la tabla completa, **sin ningún aviso**.

La forma correcta: %in%

```
codigo_coalicion %in% c(1, 2)

resultados |>
  filter(clave_entidad %in% c("01", "02", "09"))
```

`%in%` pregunta «¿está en esta lista?» y no tiene esa trampa.

Y si dentro de `filter()` escribes `=` en vez de `==`, R sí te va a detener con un error — a diferencia de la trampa de arriba, esa sí truená.

Por qué no usamos corchetes

`filter()`: de una sensación a una tabla

La trampa silenciosa

`select()`, `arrange()` y sus atajos

`mutate()` y la escala municipal

Ejercicio

Quedarte con las columnas que importan

```
resultados |> select(entidad, participacion)
```

```
resultados |> select(-clave_entidad)
```

Con `-` excluyes en vez de escoger. Sirve cuando quieres «todo menos esto» y enlistar lo demás sería más largo.

select() con un patrón, no un nombre

```
resultados |> select(starts_with("pct_"))
```

Agarra de un golpe las tres columnas de porcentaje.

Hay más atajos como este —`ends_with()`, `contains()`— en el acordeón del curso.

Dos verbos de inspección

```
resultados |> rename(part_pct = participacion)

municipios |> distinct(entidad) |> nrow()
```

`rename()` cambia un nombre sin tocar los datos. `distinct()` te dice cuántos valores únicos hay.

El segundo debe dar 32: una confirmación barata de que el archivo municipal cubre las 32 entidades.

arrange(): ordenar

```
resultados |> arrange(desc(participacion))
```

`desc()` invierte el orden. Sin él, `arrange()` ordena siempre de menor a mayor.

slice_max(): el atajo de «los primeros N»

```
resultados |> slice_max(participacion, n = 5)
```

Hace en una línea lo que `arrange()` y `head()` hacían en dos, y se lee mejor: «quédate con las 5 entidades de arriba en participación».

Por qué no usamos corchetes

`filter()`: de una sensación a una tabla

La trampa silenciosa

`select()`, `arrange()` y sus atajos

`mutate()` y la escala municipal

Ejercicio

Crear una columna nueva

```
resultados <- resultados |>  
  mutate(ventaja = pct_shh - pmax(pct_fcm, pct_mc))
```

`pmax()` compara par por par y se queda con el mayor de los dos —el segundo lugar, sea cual sea la coalición.

Cuando algo falla

Error: object 'ventaja' not found

Traducción: no existe nada que se llame así todavía.

Casi siempre es que corriste `mutate()` sin guardar el resultado: `mutate()` nunca modifica `resultados` por sí solo. Hay que reasignarlo con `resultados <- resultados |> mutate(...)`.

De la intuición a la tabla

- «Siento que Morena arrasó en el sur» es una intuición.

De la intuición a la tabla

- «Siento que Morena arrasó en el sur» es una intuición.
- `filter(ventaja > 20)` es una pregunta empírica.

De la intuición a la tabla

- «Siento que Morena arrasó en el sur» es una intuición.
- `filter(ventaja > 20)` es una pregunta empírica.

El salto

De «siento que Morena arrasó en el sur» a `filter(ventaja > 20)`: una intuición política convertida en una línea que cualquiera puede correr, revisar y refutar. Ese es el salto de hoy.

La pregunta del título, contestada

```
resultados |>  
  filter(ventaja > 20) |>  
  arrange(desc(ventaja)) |>  
  select(entidad, ventaja)
```

Esta tabla contesta, a nivel estatal, la pregunta con la que abrió la sesión.

La misma pregunta, en 2,475 municipios

```
municipios <- read_csv(here(
  "datos", "limpios",
  "presidencial_2024_municipio.csv"))

municipios |> filter(ventaja_shh > 20) |> nrow()
```

`municipios` ya trae `ventaja_shh` calculada: no tuviste que hacer la resta 2,475 veces. Eso es lo que aprendiste en la sesión 1, otra vez, a otra escala.

Por qué no usamos corchetes

`filter()`: de una sensación a una tabla

La trampa silenciosa

`select()`, `arrange()` y sus atajos

`mutate()` y la escala municipal

Ejercicio

Quince minutos, tres niveles

- **Calentamiento** — sube el umbral de 20 a 10 puntos y compara.
- **De verdad** — construye tu propia ventaja y tu propia pregunta.
- **Si te sobra tiempo** — investiga `slice_min()` y `%in%` con más de dos valores.

El archivo es `03_ejercicio.R`. La solución se publica al terminar la sesión.

Lo que ya sabemos hacer

- Cambiar los corchetes de base R por un pipe legible: `filter()`, `select()`, `arrange()` y `mutate()`.
- Reconocer la trampa `filter(var == 1 | 2)` y usar `%in%` en su lugar.
- Convertir una intuición política propia en una tabla que la pone a prueba.

LA SEMANA QUE VIENE

Agrupar, resumir, cruzar

Vamos a comparar entidades entre sí de verdad —no fila por fila, sino resumidas— y a cruzar dos bases por una clave que no siempre coincide.